

⚡ RemoteDesk Omni-System

Full-stack, bi-directional remote access across Desktop ↔ Mobile. WebRTC P2P streaming · Normalized input relay · Android & iOS native support · Electron host

Table of Contents

- [Overview](#)
 - [Architecture](#)
 - [Folder Structure](#)
 - [Prerequisites](#)
 - [Quick Start](#)
 - [Detailed Setup](#)
 - [1 · Signaling Server](#)
 - [2 · Web Dashboard](#)
 - [3 · Desktop Host](#)
 - [4 · Mobile Client](#)
 - [How It Works](#)
 - [Configuration Reference](#)
 - [Platform Notes](#)
 - [Troubleshooting](#)
 - [Security Considerations](#)
-

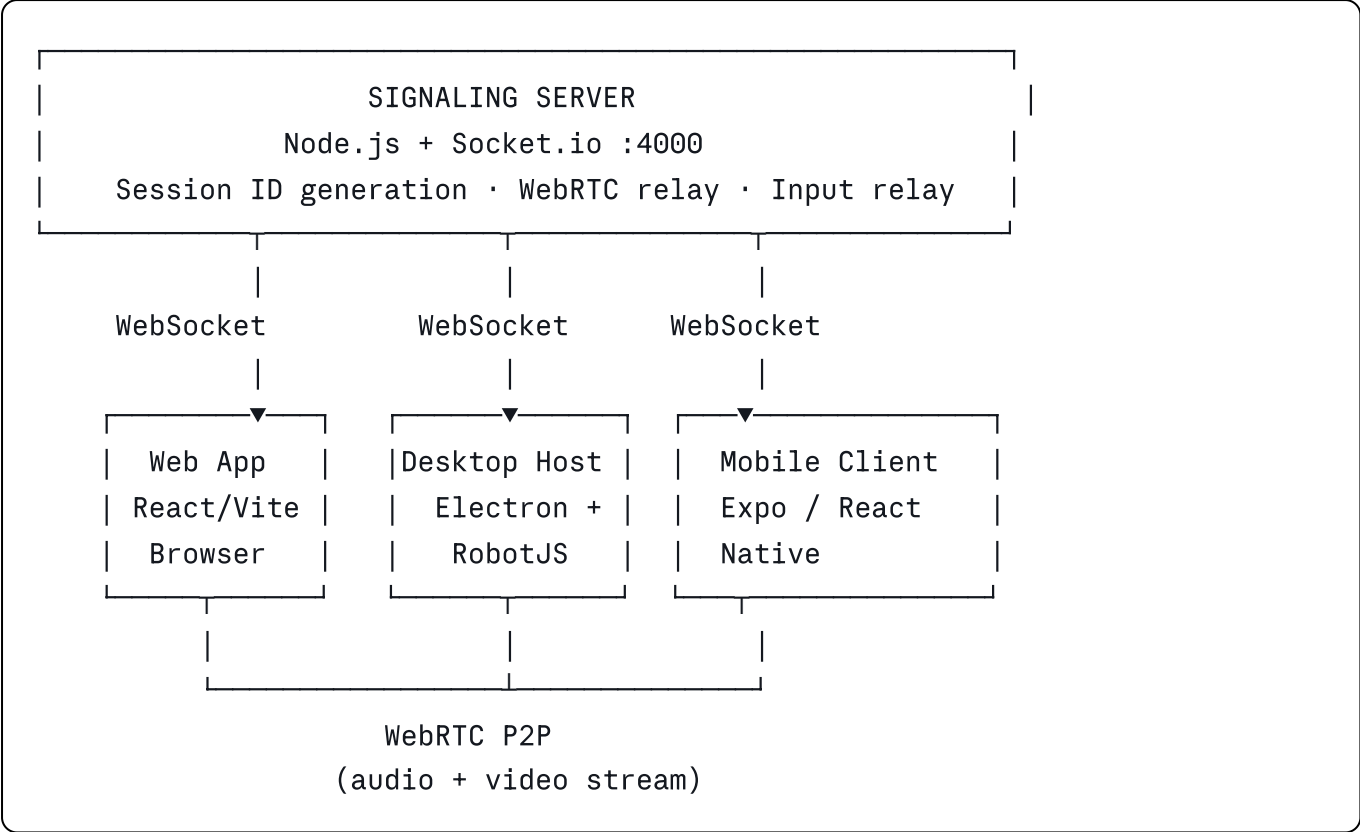
Overview

RemoteDesk is a **multi-platform remote control system** built on WebRTC and Socket.io. It supports two directions of control:

Direction	Description
Mobile → Desktop	Use your phone as a trackpad/keyboard to control a computer
Desktop → Mobile	View and control an Android or iOS device from a browser or desktop app

All input events use a **normalized coordinate system (0.0 - 1.0)** so that a tap at $(0.5, 0.5)$ always maps to the center of the remote screen, regardless of resolution differences between a 4K monitor and a 390px mobile display.

Architecture



Component Roles

Component	File	Role
Signaling Server	<code>server.js</code>	Creates/joins sessions, relays WebRTC offers/answers/ICE, forwards input events
Web Dashboard	<code>WebApp.tsx</code>	Browser-based viewer and controller with keyboard/mouse relay
Desktop Host	<code>DesktopMain.js</code>	Electron app that captures screen, executes received inputs via RobotJS
Mobile Client	<code>MobileApp.js</code>	Expo app with trackpad mode and remote viewer mode

Folder Structure

After running `Setup.sh`, your project will look like this:

```

remotedesk/
|
├─ server.js           ← Source: Signaling server (copy to server/)
├─ WebApp.tsx          ← Source: React web app (copy to
web/src/App.tsx)
├─ MobileApp.js        ← Source: Expo mobile app (copy to
mobile/App.js)
├─ DesktopMain.js      ← Source: Electron main process (copy to
desktop/)
├─ Setup.sh            ← One-shot installer script
├─ README.md           ← This file
|
├─ server/             ← Signaling Hub
|   └─ package.json
|       └─ server.js
|
├─ web/                ← React 18 + Vite Dashboard
|   └─ package.json
|   └─ vite.config.ts
|   └─ tailwind.config.js
|   └─ .env             ← Set VITE_SERVER_URL here
|   └─ index.html
|       └─ src/
|           └─ App.tsx   ← Main component (from WebApp.tsx)
|           └─ main.tsx
|           └─ index.css
|
├─ desktop/            ← Electron Desktop Host
|   └─ package.json
|   └─ DesktopMain.js   ← Electron main process
|   └─ preload.js       ← Context bridge (auto-generated by Setup.sh)
|       └─ renderer/
|           └─ index.html ← Status UI (auto-generated by Setup.sh)
|
├─ mobile/             ← Expo React Native Client
|   └─ package.json
|   └─ app.json
|   └─ babel.config.js
|   └─ tailwind.config.js
|   └─ App.js           ← Main screen (from MobileApp.js)
|       └─ SETUP_NOTES.md ← Native module instructions (Android/iOS)

```

Prerequisites

Requirement	Version	Notes
Node.js	v18 or later	Required for all environments
npm	v9+	Comes with Node.js
Expo CLI	Latest	<code>npm install -g expo-cli</code>
Xcode	14+	iOS builds only (macOS required)
Android Studio	Latest	Android builds / emulator
Python 3	3.8+	Required by <code>electron-rebuild</code> on some platforms
Build Tools	—	<code>build-essential</code> (Linux), Xcode CLI (macOS), Visual Studio (Windows) — needed for RobotJS native compilation

Install build tools by platform

macOS

```
bash
xcode-select --install
```

Ubuntu / Debian

```
bash
sudo apt-get install -y build-essential python3 libxtst-dev libpng-dev
```

Windows

```
powershell
npm install --global windows-build-tools
```

Quick Start

```
bash
```

```
# 1. Clone / place all source files in a directory
mkdir remotedesk && cd remotedesk
# (place server.js, WebApp.tsx, MobileApp.js, DesktopMain.js, Setup.sh here)

# 2. Run the installer
chmod +x Setup.sh
bash Setup.sh

# 3. Configure environment (see Configuration Reference below)
echo "VITE_SERVER_URL=http://YOUR_LAN_IP:4000" > web/.env

# 4. Start everything (4 separate terminals)
cd server && npm start           # Terminal 1 – Signaling server
cd web && npm run dev             # Terminal 2 – Web dashboard
cd desktop && npm start           # Terminal 3 – Desktop host
cd mobile && npx expo start       # Terminal 4 – Mobile (scan QR with Expo Go)
```

Detailed Setup

1 • Signaling Server

The signaling server is the hub. It must be reachable by all other components.

```
bash

cd server
npm install          # if not done by Setup.sh
npm start           # production
npm run dev         # development with auto-reload (nodemon)
```

Verify it's running:

```
bash

curl http://localhost:4000/health
# → {"status":"ok","sessions":0,"uptime":12.3}
```

Environment variables:

Variable	Default	Description
<code>PORT</code>	<code>4000</code>	Listening port

Deploying to a VPS / cloud:

```
bash

# Install PM2 for process management
npm install -g pm2
cd server
pm2 start server.js --name remotedesk-server
pm2 save
pm2 startup
```

2 • Web Dashboard

A React 18 + Vite app. Run in any modern browser.

```
bash

cd web

# Create your env file (copy from example)
cp .env.example .env
# Edit .env – set VITE_SERVER_URL to your signaling server address
nano .env
```

.env contents:

```
env

VITE_SERVER_URL=http://192.168.1.100:4000
```

Important: Use your LAN IP or public domain, not `localhost`, when connecting from other devices.

```
bash

npm run dev      # dev server at http://localhost:5173
npm run build    # production build → dist/
npm run preview  # preview production build
```

Deploy the web app (optional):

```
bash

npm run build
# Upload the dist/ folder to any static host: Vercel, Netlify, Cloudflare Pages
```



The Electron app runs on the computer you want to control. It captures the screen and executes incoming input events using RobotJS.

```
bash

cd desktop

# If native modules failed during Setup.sh, rebuild them:
npm run rebuild
# This compiles robotjs and wrtc against Electron's Node ABI

npm start          # Launch the desktop host
npm run dev         # Launch with DevTools open
npm run build       # Package as .exe / .dmg / .AppImage
```

On first launch:

- macOS will prompt for **Screen Recording** permission — approve it in System Preferences → Privacy & Security
- The app auto-connects to the server and creates a session
- A **9-digit session ID** appears in the window — share it with your mobile or browser

Native compilation notes:

- RobotJS requires Python 3, `node-gyp`, and platform build tools
- On Windows, ensure Visual Studio Build Tools are installed before `npm install`
- On Linux, `libxtst-dev` and `libpng-dev` are required

4 • Mobile Client

Built with Expo. Run on a physical device or emulator.

```
bash
```

```
cd mobile
```

```
# IMPORTANT: Set your server address first
# Edit App.js line ~19:
#   const SERVER_URL = "http://192.168.1.100:4000";

npx expo start
# → Scan the QR code with Expo Go (iOS App Store / Google Play)
# → Or press 'a' for Android emulator, 'i' for iOS simulator
```

Running a development build (required for native modules):

```
bash
```

```
npx expo run:android    # Android – requires Android Studio
npx expo run:ios        # iOS – requires Xcode on macOS
```

`react-native-webrtc` requires a **development build** (not Expo Go) for full functionality. The app UI works in Expo Go for testing layout, but WebRTC and native input modules need a compiled build.

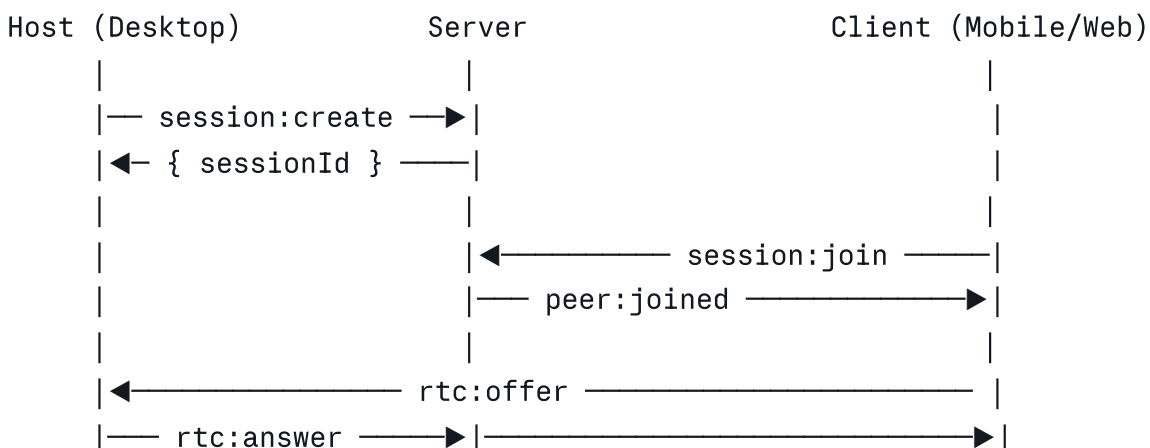
Building for distribution:

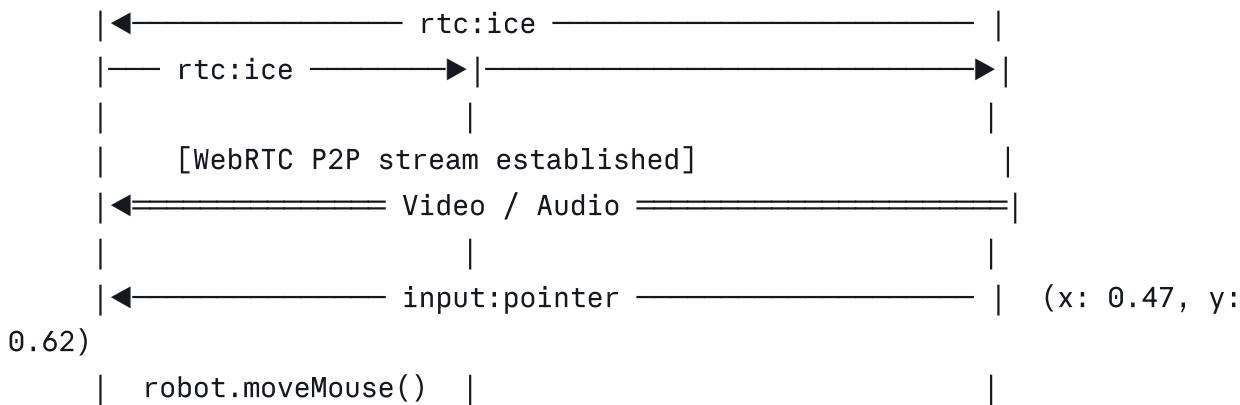
```
bash
```

```
# Using EAS Build (Expo Application Services)
npm install -g eas-cli
eas build --platform android  # APK / AAB
eas build --platform ios      # IPA
```

How It Works

Session Lifecycle





Coordinate Normalization

All pointer events use a **0.0-1.0 normalized space**:

Mobile screen (390×844)	Desktop screen (3840×2160)
tap at pixel (185, 522)	maps to pixel (1805, 2528)
$\text{normX} = 185 / 390 = 0.474$	$\text{px} = 0.474 \times 3840 = 1820$
$\text{normY} = 522 / 844 = 0.619$	$\text{py} = 0.619 \times 2160 = 1337$

The server validates that `x` and `y` are within `[0, 1]` before relaying. The desktop host multiplies by actual screen dimensions (including DPI scale factor for HiDPI/Retina displays).

Session Expiry

Sessions auto-expire after **30 minutes of inactivity**. Any input event, ICE candidate, or peer join resets the timer. When a session expires, all connected sockets receive a `session:expired` event and are disconnected.

Configuration Reference

Server (`server/server.js`)

Constant	Default	Description
<code>PORT</code>	<code>4000</code>	HTTP + Socket.io port
<code>SESSION_TTL_MS</code>	<code>1800000</code>	Session idle timeout (30 min)
<code>HEARTBEAT_MS</code>	<code>25000</code>	Socket.io ping interval
<code>MAX_SESSIONS</code>	<code>1000</code>	Max concurrent sessions

Web App (`web/.env`)

Variable	Example	Description
<code>VITE_SERVER_URL</code>	<code>http://192.168.1.10:4000</code>	Signaling server URL

Desktop Host (`desktop/DesktopMain.js`)

Constant	Default	Description
<code>SERVER_URL</code>	<code>http://localhost:4000</code>	Override with <code>REMOTEDesk_SERVER</code> env var
<code>RECONNECT_ATTEMPTS</code>	<code>10</code>	Socket.io reconnection attempts
<code>RECONNECT_DELAY_MS</code>	<code>3000</code>	Delay between reconnect attempts

Mobile App (`mobile/App.js`)

Constant	Default	Description
<code>SERVER_URL</code>	<code>http://your-server-ip:4000</code>	Must be set before building
<code>SENSITIVITY</code>	<code>1.8</code>	Trackpad cursor speed multiplier

Platform Notes

Android — Accessibility Service

The `AccessibilityModule` native module allows RemoteDesk to inject taps and swipes at the OS level via `dispatchGesture`, without root access.

Setup steps:

- Create `android/app/src/main/java/com/remotedesk/AccessibilityModule.java`
- Extend `AccessibilityService`; implement `onAccessibilityEvent` and `dispatchGesture`
- Declare in `AndroidManifest.xml`:

```
xml
```

```
<service android:name=".AccessibilityModule"
    android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">
    <intent-filter>
        <action android:name="android.accessibilityservice.AccessibilityService"
    </intent-filter>
</service>
```

4. Register as a React Native module in `MainApplication.java`
5. The user must manually enable RemoteDesk in **Settings → Accessibility**

See `mobile/SETUP_NOTES.md` for the complete Java skeleton.

iOS — ReplayKit + ScreenControl

Screen capture (host mode):

- Uses `RPBroadcastActivityViewController` (ReplayKit) to start a broadcast upload extension
- The extension pushes `CMSampleBuffer` frames into the WebRTC pipeline via a Swift bridge
- Requires a **Broadcast Upload Extension** target in Xcode

Input injection (client mode):

- Consumer apps: taps are simulated via `UIAccessibility` (limited)
- MDM/Enterprise apps: uses the `XCTest` / ScreenControl private framework
- `ScreenControlModule.swift` bridges Swift → React Native via `RCTBridgeModule`

Required Xcode config:

- Add entitlement: `com.apple.developer.screen-capture`
- Info.plist keys: `NSCameraUsageDescription`, `NSMicrophoneUsageDescription`
- Broadcast extension: `RPBroadcastSampleHandler` subclass

macOS — Screen Recording Permission

On first launch of the Desktop Host, macOS will show a permission dialog for Screen Recording. If it doesn't appear automatically:

1. Open **System Settings → Privacy & Security → Screen Recording**
2. Enable the toggle for **RemoteDesk**
3. Restart the app

Linux — Screen Capture

On Wayland, `desktopCapturer` requires a portal backend:

```
bash
sudo apt-get install xdg-desktop-portal xdg-desktop-portal-wlr
```

On X11, it works out of the box.

Troubleshooting

"Session not found" when joining

The session ID is case-sensitive and must be exactly 9 digits. Sessions expire after 30 minutes of inactivity. Ask the host to create a new session.

WebRTC stream not appearing

- Ensure both peers are on the same network or the server has a public IP
- Check that no firewall blocks UDP traffic (WebRTC uses UDP for media)
- For production, deploy a **TURN server** (e.g., coturn) for NAT traversal:

```
javascript
// Add to ICE_SERVERS in all components:
{ urls: "turn:your-turn-server.com:3478", username: "user", credential: "pass
```

RobotJS fails to install / compile

```
bash
# Ensure build tools are present, then:
cd desktop
npm install --build-from-source robotjs
npm run rebuild
```

On macOS, if you see Python errors:

```
bash
npm config set python python3
```

electron-rebuild fails

```
bash

cd desktop
./node_modules/.bin/electron-rebuild --force --types prod,dev --module-dir .
```

Mobile app can't connect to server

- Confirm the `SERVER_URL` in `App.js` is your **LAN IP** (not `localhost`)
- Ensure your phone and server are on the same Wi-Fi network
- Check the server is running: `curl http://YOUR_IP:4000/health`
- Disable VPNs / proxies on the phone during testing

High CPU on Desktop Host (screen capture)

The default capture loop runs at 30fps using Electron's `desktopCapturer`. For lower CPU usage, reduce the interval:

```
javascript

// In DesktopMain.js, change:
}, 1000 / 30); // 30fps
// To:
}, 1000 / 15); // 15fps
```

For production, consider replacing the `desktopCapturer` polling loop with a native screen capture library (e.g., `screen-capture-recorder` on Windows, `CoreGraphics` stream on macOS).

Security Considerations

Risk	Mitigation
Anyone with the session ID can join	IDs are 9-digit crypto-random (900M combinations); add a PIN layer for production
Input events relayed without auth	Server validates coordinate bounds; add per-session auth tokens for production
WebRTC streams are P2P encrypted	DTLS-SRTP is mandatory in all compliant browsers and the <code>wrtc</code> library

Risk	Mitigation
Clipboard sync exposes sensitive data	100KB payload cap enforced server-side; scope to explicit user action
Session persistence	Sessions auto-expire after 30 min idle; <code>session:expired</code> notifies all peers

For production deployments:

- Run the signaling server behind HTTPS (use a reverse proxy like Nginx + Let's Encrypt)
- Replace `*` CORS origin with your actual domain
- Add rate limiting to Socket.io events
- Implement session passwords or short-lived join tokens

License

MIT — Use freely, contribute back.

Built with Node.js · Socket.io · React · Electron · Expo · WebRTC · RobotJS